CMOS RISC-V — Relatório técnico da Fase 2

Versão preliminar 0.4 — 17 de outubro de 2026

Equipe: CMOS. Integrantes, instituição e orientador: **a preencher**. Repositório: cmosuea-arch/RV32-multicycle. RTL da captura: `7ddc6bf`. Estrutura conforme os tópicos 1–6 do Guia de Submissão [R1].

1. Visão Geral do Projeto

1.1 Objetivo e arquitetura

Implementar em SystemVerilog um processador RISC-V de 32 bits, multiciclo, conforme o Block Guide ChampionCHIP: RV32I, Zmmul e Xicrc, com meta de 47 instruções. A implementação é própria, sem incorporar o core Ibex.

`riscv_core` integra PC, IR, banco de registradores, imediatos, decoder, ALU, comparador de branch, multiplicador, LSU e controle. `riscv_top` conecta o núcleo a IMEM, DMEM e decoder de endereços. A configuração padrão verifica **41/47 instruções (87,23%)**. CRCB/CRCH/CRCW estão implementadas sob contrato experimental CMOS, habilitado por `EXPERIMENTAL_XICRC=1`; a confirmação oficial está pendente.

A execução segue `FETCH → DECODE → EXECUTE → WRITEBACK`. Loads/stores acrescentam MEMORY para a DMEM síncrona. O PC atualiza na aposentadoria; reset síncrono ativo alto, na borda de subida, reinicia em `0x00400000`. Acessos inválidos ou instruções não suportadas levam a FAULT até reset, sem implementar traps privilegiados.

Memória	Base	Capacidade padrão	simple_system experimental
IMEM, leitura combinacional	0x00400000	4 MiB	2 KiB
DMEM, acesso síncrono	0x10010000	8 KiB	1 KiB

A organização de bytes é little-endian. A IMEM também fornece constantes; a DMEM aceita escrita por byte e não é apagada por reset. `simple_system` usa o CRC provisório e saídas done/success/fault para verificar firmware próprio. Pinos/periféricos e implementação das memórias do top final seguem pendentes.

1.2 Desenvolvimento e ferramentas

O desenvolvimento parte dos contratos de interface, reset e memória. Cada bloco passa por implementação, TB autoverificável, integração e regressão antes do próximo incremento. Marcos: **21/47** (aritmética), **29/47** (controle de fluxo), **37/47** (memória) e **41/47** (Zmmul), identificados pelas tags `isa-21-v1`, `isa-29-v1`, `isa-37-v1` e `isa-41-v1`.

Python gera referências, vetores e programas; Icarus compila/simula RTL e TB; GCC/binutils geram o firmware RV32I_Zmmul/ILP32; Verilator realiza lint. Make coordena os comandos. Logs e hashes SHA-256 identificam o conteúdo verificado. Os vetores são determinísticos; firmware compilado complementa os programas gerados pelo modelo Python [R2, R4].

A rodada completa de 13/10/2026 (Manaus) aprovou 31 TBs e lint. Em 17/10 foi reexecutado apenas o firmware próprio para capturar waveform. Síntese genérica e simulação de netlist também passaram; firmware oficial e fluxo físico do processador ainda aguardam conclusão [R3–R6].

2. Instruções Suportadas

2.1 Cobertura e lista por categoria

Categoria	Esperadas	Implementadas	Cobertura	Instruções da baseline
Arithmetic and Logic (Register)	10	10	100%	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
Arithmetic and Logic (Immediate)	9	9	100%	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI
Load	5	5	100%	LB, LH, LW, LBU, LHU
Store	3	3	100%	SB, SH, SW
Branch	6	6	100%	BEQ, BNE, BLT, BGE, BLTU, BGEU
Jump	2	2	100%	JAL, JALR
Upper Immediate	2	2	100%	LUI, AUIPC
Sistema / Sincronização	3	0	0%	Definições oficiais pendentes
Multiplication	4	4	100%	MUL, MULH, MULHSU, MULHU
CRC	3	0	0%	Confirmação oficial pendente
Total	**47**	**41**	**87,23%**	**Baseline integrada e verificada**

Perfil experimental: três operações adicionais, CRCB/CRCH/CRCW, passam no contrato CMOS. A cobertura de conformidade permanece 41/47 até a confirmação oficial do algoritmo. Nenhuma instrução foi validada pelo firmware oficial.

A contagem exige decode, execução integrada e regressão, com evidência por instrução em [R3]. Os três nomes/códigos de sistema não foram presumidos apenas pela quantidade indicada no guia.

2.2 Convenções das codificações

Os campos das tabelas seguintes estão em binário. O traço indica campo não usado como discriminador; seus bits podem fazer parte do imediato. Em shifts imediatos, funct7 discrimina a operação e shamt tem cinco bits. PASS_B é uma operação interna da ALU, sem acrescentar instrução à cobertura.

As 41 codificações da baseline e as três experimentais foram conferidas com `docs/isa/coverage.csv`. A matriz contém testes, firmware oficial, marco Git e caminho da evidência por instrução; versões Markdown/Excel acompanham o CSV.

2.3 Codificação — aritmética e imediatos

Instrução	Opcode [6:0]	Funct3 [14:12]	Funct7 [31:25]
ADD	0110011	000	0000000
SUB	0110011	000	0100000
SLL	0110011	001	0000000
SLT	0110011	010	0000000
SLTU	0110011	011	0000000
XOR	0110011	100	0000000
SRL	0110011	101	0000000
SRA	0110011	101	0100000
OR	0110011	110	0000000
AND	0110011	111	0000000
ADDI	0010011	000	—
SLTI	0010011	010	—
SLTIU	0010011	011	—
XORI	0010011	100	—
ORI	0010011	110	—
ANDI	0010011	111	—
SLLI	0010011	001	0000000
SRLI	0010011	101	0000000
SRAI	0010011	101	0100000
LUI	0110111	—	—
AUIPC	0010111	—	—

SLT/SLTI comparam com sinal; SLTU/SLTIU, sem sinal. O imediato I é estendido com sinal inclusive em SLTIU. Deslocamentos por registrador usam rs2[4:0]. LUI forma o imediato U; AUIPC soma esse imediato ao PC da instrução.

2.4 Codificação — memória, controle de fluxo e multiplicação

Instrução	Opcode [6:0]	Funct3 [14:12]	Funct7 [31:25]
BEQ	1100011	000	—
BNE	1100011	001	—
BLT	1100011	100	—
BGE	1100011	101	—
BLTU	1100011	110	—
BGEU	1100011	111	—
JAL	1101111	—	—
JALR	1100111	000	—
LB	0000011	000	—
LH	0000011	001	—
LW	0000011	010	—
LBU	0000011	100	—
LHU	0000011	101	—
SB	0100011	000	—
SH	0100011	001	—
SW	0100011	010	—
MUL	0110011	000	0000001
MULH	0110011	001	0000001
MULHSU	0110011	010	0000001
MULHU	0110011	011	0000001

2.5 Codificação — CRC experimental

Instrução	Opcode [6:0]	Funct3 [14:12]	Funct7 [31:25]
CRCB	0110011	000	1000000
CRCH	0110011	001	1000000
CRCW	0110011	010	1000000

O guia confirma esses códigos, mas não detalha todo o algoritmo. O contrato CMOS usa CRC-16/0x1021, acumulador em rs1[15:0], 1/2/4 bytes baixos de rs2 em ordem little-endian, MSB primeiro em cada byte, sem reflexão/XOR final e resultado estendido com zeros. A semente vem do software. Exemplo: ASCII 123456789, seed `0xffff`, resultado `0x29b1` [R2, R3].

3. Datapath

3.1 Operandos, execução e writeback

Os três painéis foram conferidos com o RTL em 17/10/2026. Nomes iguais identificam a mesma rede entre blocos/painéis. Fontes vetoriais: [R2].

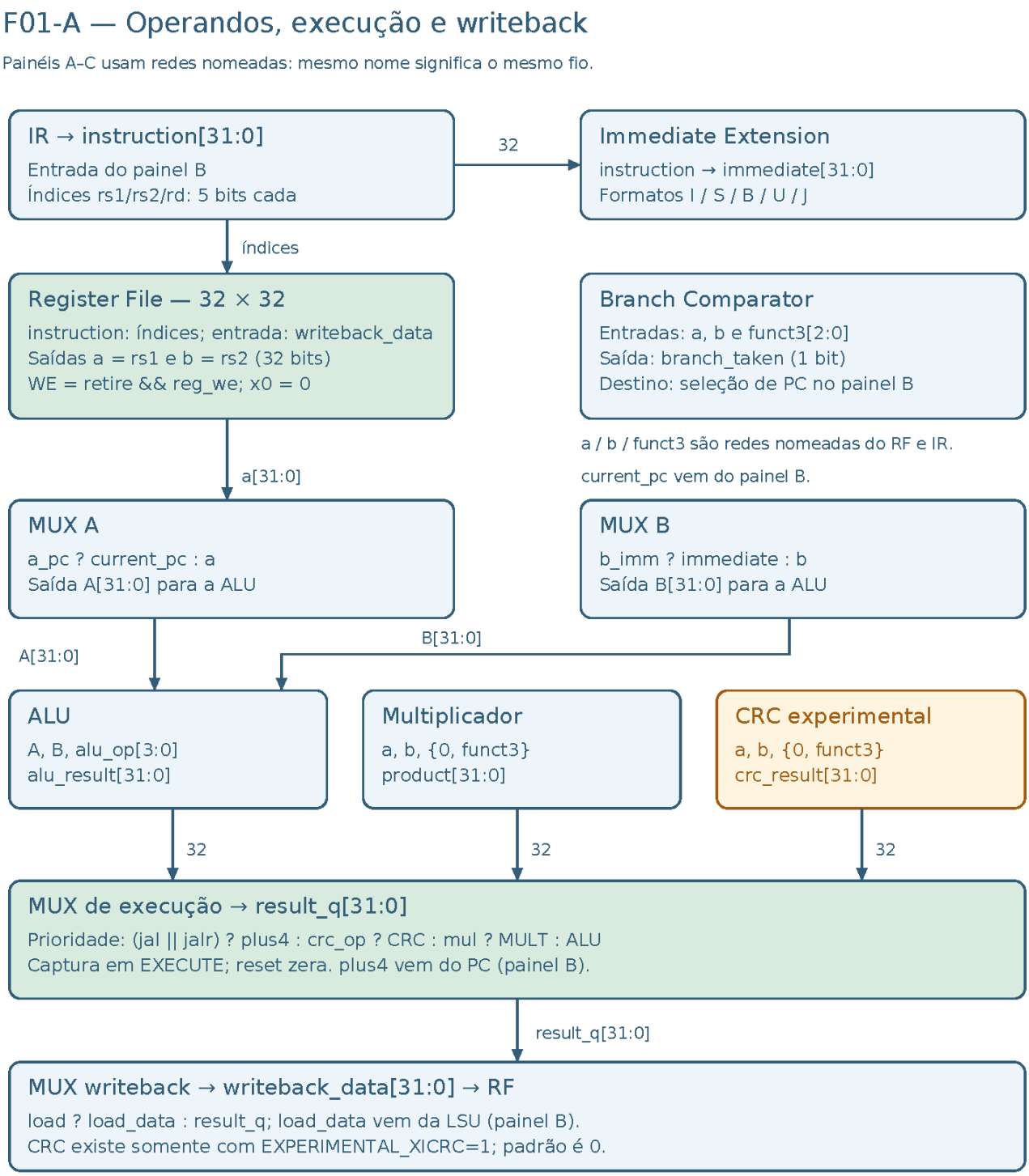


Figura F01-A. Muxes selecionam operandos e resultado; result_q captura em EXECUTE; writeback escolhe resultado ou load. CRC é opcional e experimental.

3.2 Próximo PC, LSU e memórias

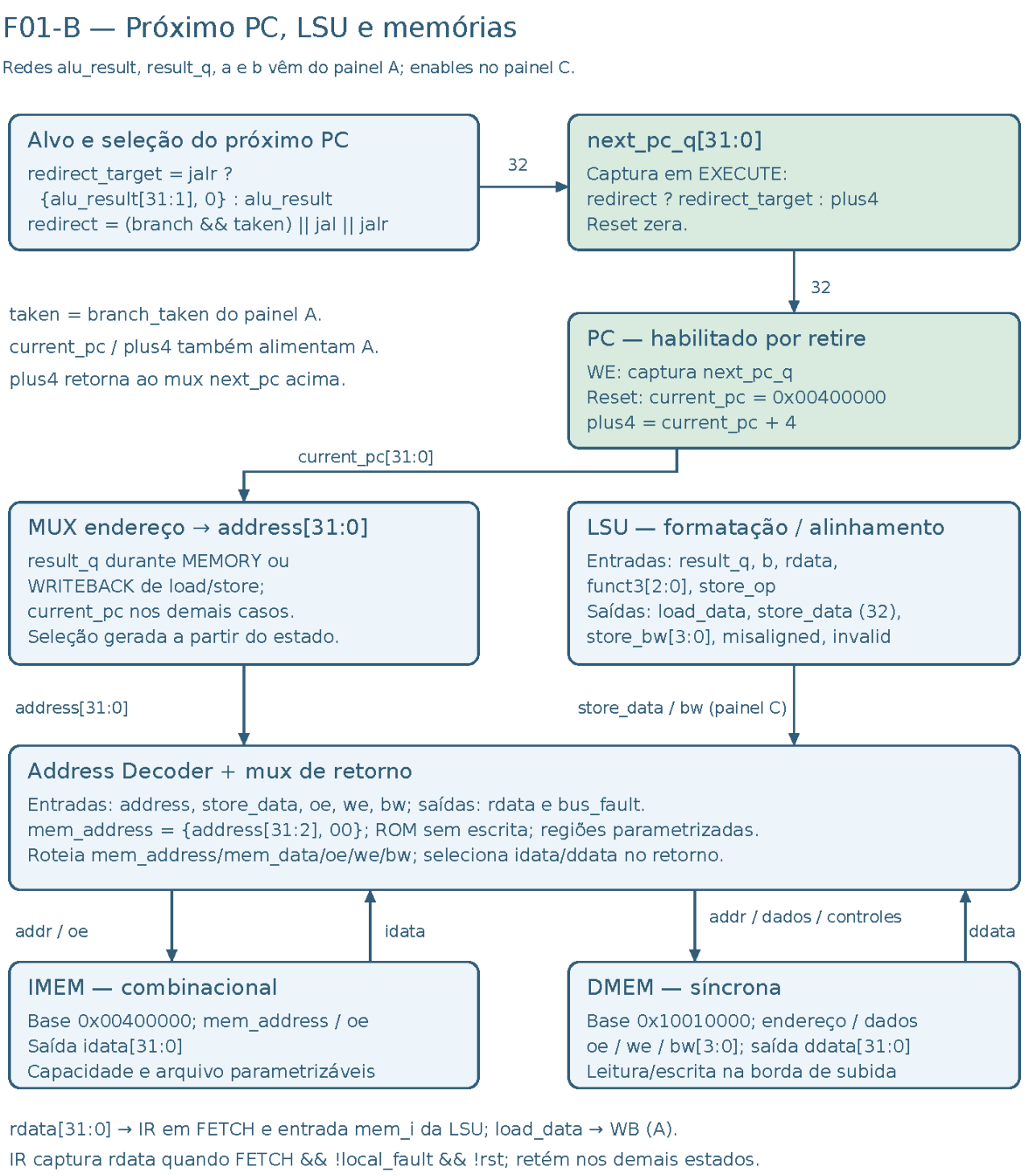


Figura F01-B. O alvo de JALR tem bit 0 zerado. next_pc_q captura o próximo endereço; PC atualiza em retire. A LSU formata loads/stores e gera byte write. O decoder seleciona a memória e protege a ROM. Endereço de dados é conservado em MEMORY e WRITEBACK de load/store; o store não repete escrita em WRITEBACK.

3.3 Controle e sincronização

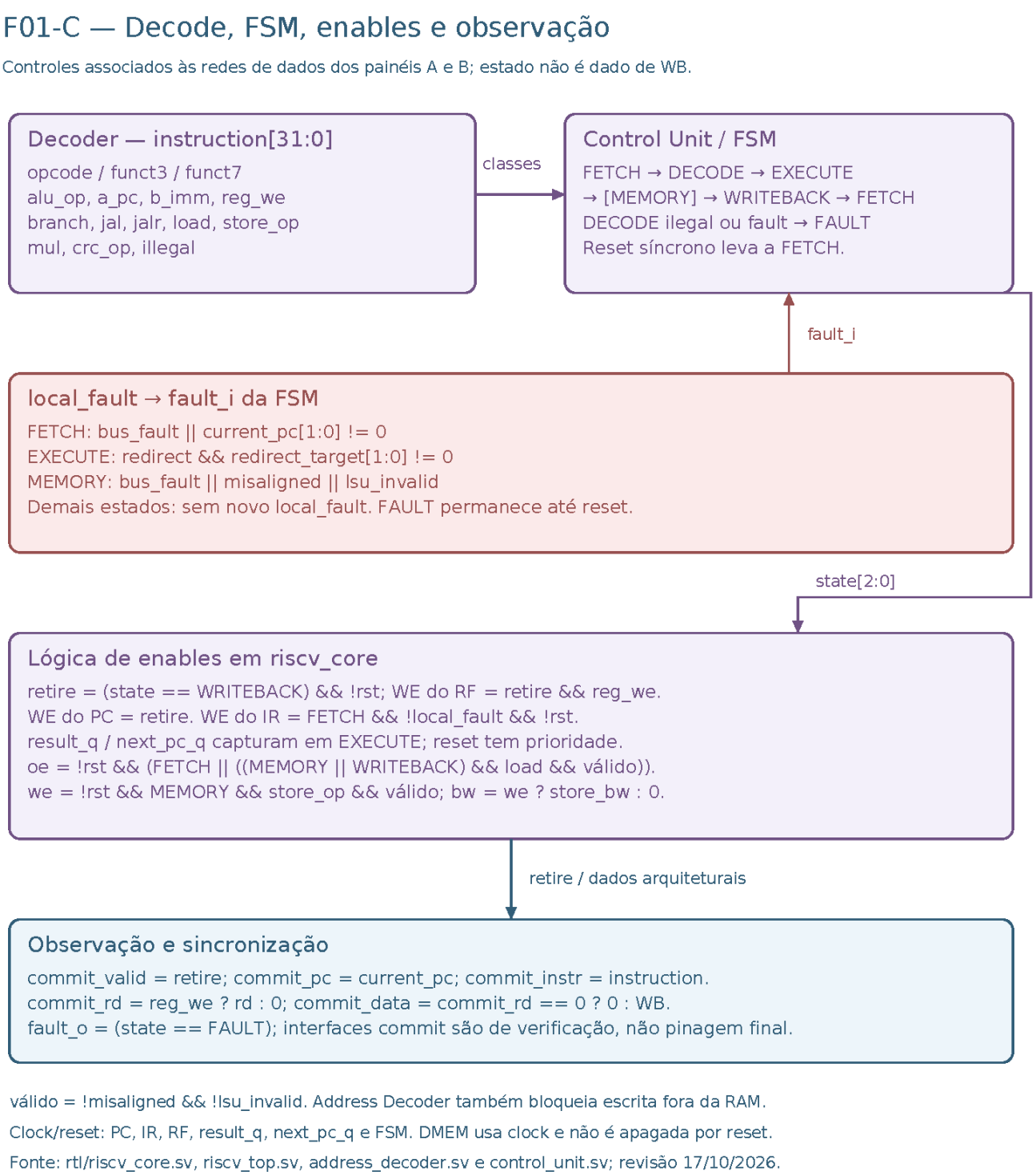


Figura F01-C. Decoder gera seleções/classes; FSM e lógica do core geram enables e oe/we/bw. Instruções sem acesso a dados usam quatro ciclos; loads/stores, cinco. FAULT é diagnóstico CMOS até reset. commit_* observa a aposentadoria para os TBs; não define a pinagem final.

4. Módulos e Testbenches

4.1 Datapath básico e multiplicação

Arquivos: `rtl/<módulo>.sv`, `docs/modules/<módulo>.md` e bancadas em `tb/`. Dados são de 32 bits; índices RF, de 5 bits. Os testes abaixo possuem resultados PASS na rodada identificada em [R4]. Os diagramas ChipInventor por módulo serão incorporados após sua entrega pelo colaborador responsável.

Módulo	Função	TB principal	Casos verificados
alu	11 operações combinacionais; seletor de 4 bits	tb_alu	18.304 vetores; extremos, overflow modular, shifts e signed/unsigned
regfile	32×32; duas leituras combinacionais, escrita síncrona; x0 fixo	tb_regfile	Escrita/leitura, 1.024 pares, x0, enable e reset
imm_ext	Reconstrução e extensão dos formatos I/S/B/U/J	tb_imm_ext	5.102 vetores; extremos, bits e sinal
pc	Endereço atual e PC+4; atualização habilitada e reset	tb_pc	Sequência, retenção, alvos, wrap e reset
ir	Captura habilitada em FETCH e retenção da instrução	tb_ir	Captura/retenção, sequência e reset
imem	ROM combinacional parametrizável; arquivo completo ou zeros	tb_imem	Conteúdo, enable, alinhamento e limites
branch_comp	Igualdade e ordem signed/unsigned conforme funct3	tb_branch_comp	9.152 vetores; limites e condições tomadas/não tomadas no core
multiplier	Partes baixa/alta de produtos signed, mixed e unsigned	tb_multiplier	18.304 vetores; zero, -1, extremos e quatro operações Zmmul

Integrações progressivas: `tb_rf_alu`, `tb_ir_imm`, `tb_fetch`, `tb_pc_alu`, `tb_branch_pc`, `tb_datapath` e `tb_mult_rf`. Elas verificam conexões e enables antes da execução completa no core. A especificação detalhada de entradas, saídas, dependências e casos especiais permanece em [R2].

Diagramas a inserir nesta seção: F02–F08 e F14, produzidos no ChipInventor, com pinos e larguras legíveis. Os painéis podem agrupar módulos relacionados.

4.2 Memória, controle e integração

Módulo	Função	TB principal	Casos verificados
lsu	Seleção/extensão de loads; posicionamento/máscara de stores	tb_lsu	8.960 vetores; byte/half/word, offsets, sinal e alinhamento
dmem	Leitura/escrita síncronas; preserva bytes não selecionados	tb_dmem	256 combinações posição/máscara, read-first e limites
address_decoder	Seleção IMEM/DMEM, retorno, controles e proteção da ROM	tb_address_decoder	768 combinações faixa/pedido/máscara
decoder	Decodifica opcode/funct3/funct7; CRC opt-in	tb_decoder	131.072 combinações de campos por perfil; válidos e inválidos
control_unit	FSM FETCH / DECODE / EXECUTE / MEMORY / WRITEBACK / FAULT	tb_control_unit	Transições, classes, falhas e reset
crc	CRC-16 combinacional experimental; 1/2/4 bytes	tb_crc	18.304 vetores e valor conhecido 29b1
riscv_core	Integra execução, controle e barramento compartilhado	tb_core_smoke; tb_isa	PC, instrução, destino, dado e RF por aposentadoria
riscv_top	Liga core, decoder de endereços e memórias	tb_firmware; tb_memory_faults	Firmware, acessos e bloqueio de efeitos inválidos
simple_system	Sistema pequeno com CRC e saídas de término/sucesso/falha	tb_simple_system	Sete cenários de boot/reset/assinatura/falha

Resultado agregado: 14 TBs unitários + 17 de integração e lint PASS em 13/10/2026 (Manaus). `tb_isa`: 76.409 aposentadorias, 45 stores e 12 bytes finais comparados. CRC integrado: 2.127 instruções CRC em 32.147 aposentadorias. A simulação da netlist é adicional às 31 bancadas RTL [R4, R5].

Python gera referências determinísticas; o TB encerra com erro em divergência. O executor rejeita falha, timeout, diagnóstico de compilação ou ausência de PASS. Verilator verifica largura/latches/loops sem suprimir essas classes. Manifestos identificam cada rodada por SHA-256. São testes funcionais, sem prova formal ou alegação de cobertura exaustiva de código/operandos.

Diagramas a inserir: F09–F13 e F15–F18. O diagrama CRC deve indicar o contrato provisório e o simple_system deve ser identificado como experimento.

5. Resultados OpenLane

Fluxo físico do processador pendente. O ambiente LibreLane/SKY130A foi verificado com um exemplo até GDSII; os resultados desse exemplo não representam o CMOS RISC-V [R5]. Preencher a tabela com uma única execução final identificada.

Campo	Resultado do processador
Commit RTL, top e configuração	Pendente
Ferramentas/PDK, clock e restrições	Pendente
Área total e densidade final	Não medidas
Timing, DRC/LVS e antena	Não avaliados
GDSII, GL, config.json e imagem F19	Pendentes

Síntese preparatória: simple_system passou no Yosys 0.39+124 (d73f71e81), com 32.922 células genéricas e dois boots simulados na netlist. ROM virou constantes; RAM, registradores/muxes. A inicialização da IMEM foi corrigida para selecionar arquivo OU zeros após detectar ROM zerada na síntese. O teste da netlist confirmou a correção. Não há área física ou timing medidos nessa rodada [R5].

6. Firmware Demo

6.1 Firmware próprio e evidência disponível

`start.S` configura a pilha, copia .data, limpa .bss e chama `main.c`; `link.ld` posiciona código/dados. GCC/binutils geram RV32I_Zmmul/ILP32. O programa testa boot, dados, pilha, chamadas, loops, memória e multiplicação. O startup escreve o retorno em `0x10011ff0`; o TB confere a assinatura e a palavra da RAM.

Execução isolada de 17/10/2026, RTL `7ddc6bf`, com VCD/logs/hashes em [R6]:

```
PASS firmware: signature 600dc0de, 598 retirements, 2579 cycles
```

O experimento simple_system também executa firmware próprio com os três CRCs; usa assinatura em `0x100103f0`, ROM 2 KiB e RAM 1 KiB, e passou em RTL/netlist [R5].

6.2 Firmware oficial

Pendente de recebimento e execução. Registrar versão/hash do programa, commit RTL, protocolo de sucesso, etapa atingida, log sem erros e waveform F21. Os resultados próprios são evidência complementar. Pendências de Xicrc, sistema/sincronização e interface final estão registradas em [R2].

6.3 Waveform do firmware próprio

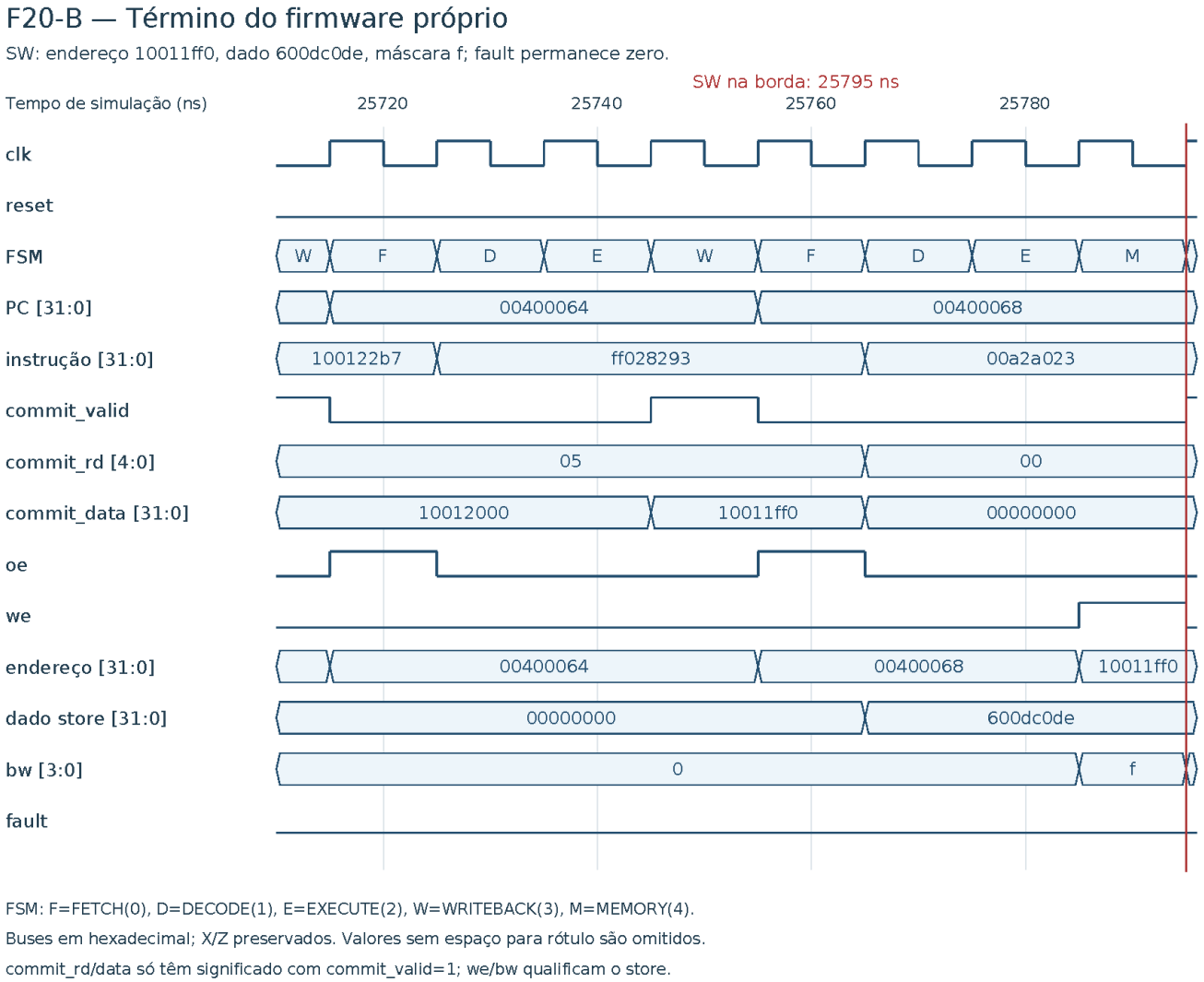


Figura F20-B — Janela de 25.710–25.796 ns da captura de 17/10/2026. SW `0x00a2a023`, no PC `0x00400068`, grava `0x600dc0de` em `0x10011ff0`. Na borda de **25.795 ns**, MEMORY tem `we=1` e `bw=0xf`; fault permanece zero. O TB confirma a RAM e termina 1 ns depois. As 598 aposentadorias precedem a escrita; o SW final ainda não foi aposentado quando a simulação encerra.

O traçado vem das mudanças reais do VCD, com buses em hexadecimal e escala 1 ps/tick; clock do TB de 10 ns. A verificação usa os valores anteriores à borda, antes das atualizações NBA. Isso não mede frequência máxima física. A imagem de boot F20-A, o VCD completo e as instruções de reprodução estão em [R6], preservados como apoio; o PDF mantém a janela de término.

6.4 Referências e evidências

[R1] ChampionCHIP, 2026, versão 1.0. Guia de Submissão e Block Guide, páginas físicas 2–4 e 6–23. Limite: PDF de 20 páginas. Hash e conferência dos requisitos: conformidade.md.

[R2] Arquitetura e método. AGENTS.md, interfaces, mapa, decisões, pendências oficiais, módulos, datapath e fontes das figuras.

[R3] ISA. Matriz CSV, versão legível, contrato CRC e matriz experimental.

[R4] Verificação. Alcance dos testes, manifesto da rodada completa, evidências CRC, Makefile, roteiro e laboratório.

[R5] Síntese e ambiente físico. Registro simple_system, resumo Yosys, log da netlist e smoke do ambiente.

[R6] Captura própria, 17/10/2026. Registro, snapshot/hashes, verificação do VCD, log e VCD comprimido.

Os manifestos comprovam os conteúdos das respectivas rodadas. O fechamento da entrega deve vincular código, testes, firmware e resultados físicos ao commit final. Autoria, diagramas ChipInventor, resultados físicos e validação oficial ainda devem ser incorporados e revisados antes da submissão.